

Day 10 - MySQL (Console Transcript)

```
mysql> USE PFS4
Database changed
mysql> CREATE TABLE Projects (
->   id INT,
->   name VARCHAR(100),
->   assignedto VARCHAR(100),
->   start DATE,
->   end DATE
-> );
Query OK, 0 rows affected (0.09 sec)

mysql>
mysql> INSERT INTO Projects VALUES
-> (1, 'Website Revamp', 'Alice', '2024-01-15', '2024-03-01'),
-> (2, 'Mobile App', 'Bob', '2024-02-01', NULL),
-> (3, 'Data Migration', NULL, NULL, NULL),
-> (4, 'SEO Optimization', 'Eve', '2024-03-10', '2024-04-20'),
-> (5, 'Cloud Migration', NULL, '2024-03-15', NULL);
Query OK, 5 rows affected (0.02 sec)
Records: 5  Duplicates: 0  Warnings: 0

mysql> SELECT*FROM PROJECTS;
+-----+-----+-----+-----+-----+
| id | name | assignedto | start | end |
+-----+-----+-----+-----+
| 1 | Website Revamp | Alice | 2024-01-15 | 2024-03-01 |
| 2 | Mobile App | Bob | 2024-02-01 | NULL |
| 3 | Data Migration | NULL | NULL | NULL |
| 4 | SEO Optimization | Eve | 2024-03-10 | 2024-04-20 |
| 5 | Cloud Migration | NULL | 2024-03-15 | NULL |
+-----+-----+-----+-----+
5 rows in set (0.01 sec)

mysql> SELECT NAME,ASSIGNEDTO WHERE ASSIGNEDTO=NULL';
ERROR 1054 (42S22): Unknown column 'NAME' in 'field list'
mysql> SELECT * FROM PROJECTS WHERE ASSIGNEDTO=NULL';
Empty set (0.01 sec)

mysql> SELECT * FROM PROJECTS WHERE ASSIGNEDTO IS 'NULL';
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near ''NULL'' at line 1
mysql> SELECT * FROM PROJECTS WHERE ASSIGNEDTO IS NULL;
+-----+-----+-----+-----+-----+
| id | name | assignedto | start | end |
+-----+-----+-----+-----+
| 3 | Data Migration | NULL | NULL | NULL |
| 5 | Cloud Migration | NULL | 2024-03-15 | NULL |
+-----+-----+-----+-----+
2 rows in set (0.01 sec)

mysql> SELECT * FROM PROJECTS WHERE START IS NULL;
+-----+-----+-----+-----+-----+
| id | name | assignedto | start | end |
+-----+-----+-----+-----+
| 3 | Data Migration | NULL | NULL | NULL |
+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql> SELECT * FROM PROJECTS WHERE END IS NOT NULL;
+-----+-----+-----+-----+-----+
| id | name | assignedto | start | end |
+-----+-----+-----+-----+
| 1 | Website Revamp | Alice | 2024-01-15 | 2024-03-01 |
| 4 | SEO Optimization | Eve | 2024-03-10 | 2024-04-20 |
+-----+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> SELECT * FROM PROJECTS WHERE ASSIGNEDTO IS NOT NULL;
+-----+-----+-----+-----+-----+
| id | name | assignedto | start | end |
+-----+-----+-----+-----+
| 1 | Website Revamp | Alice | 2024-01-15 | 2024-03-01 |
| 2 | Mobile App | Bob | 2024-02-01 | NULL |
| 4 | SEO Optimization | Eve | 2024-03-10 | 2024-04-20 |
+-----+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> UPDATE PROJECTS SET ASSIGNEDTO = 'FRANK' WHERE ASSIGNEDTO IS NULL;
Query OK, 2 rows affected (0.01 sec)
Rows matched: 2  Changed: 2  Warnings: 0
```

```
mysql> UPDATE PROJECTS SET WHERE START=NOW WHERE START IS NULL;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'WHERE START=NOW WHERE START IS NULL' at line 1
mysql> UPDATE PROJECTS SET WHERE START=NOW() WHERE START IS NULL;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'WHERE START=NOW() WHERE START IS NULL' at line 1
mysql> UPDATE PROJECTS SET START = NOW() WHERE START IS NULL;
Query OK, 1 row affected, 1 warning (0.01 sec)
Rows matched: 1 Changed: 1 Warnings: 1
```

```
mysql> SELECT*FROM PROJECTS;
+-----+-----+-----+-----+-----+
| id | name | assignedto | start | end |
+-----+-----+-----+-----+
| 1 | Website Revamp | Alice | 2024-01-15 | 2024-03-01 |
| 2 | Mobile App | Bob | 2024-02-01 | NULL |
| 3 | Data Migration | FRANK | 2026-07-23 | NULL |
| 4 | SEO Optimization | Eve | 2024-03-10 | 2024-04-20 |
| 5 | Cloud Migration | FRANK | 2024-03-15 | NULL |
+-----+-----+-----+-----+
5 rows in set (0.00 sec)
```

```
mysql> UPDATE PROJECTS SET END='COMPLETED' WHERE END IS NOT NULL;
ERROR 1292 (22007): Incorrect date value: 'COMPLETED' for column 'end' at row 1
mysql> UPDATE PROJECTS SET NAME='COMPLETED' WHERE END IS NOT NULL;
Query OK, 2 rows affected (0.01 sec)
Rows matched: 2 Changed: 2 Warnings: 0
```

```
mysql> SELECT*FROM PROJECTS;
+-----+-----+-----+-----+-----+
| id | name | assignedto | start | end |
+-----+-----+-----+-----+
| 1 | COMPLETED | Alice | 2024-01-15 | 2024-03-01 |
| 2 | Mobile App | Bob | 2024-02-01 | NULL |
| 3 | Data Migration | FRANK | 2026-07-23 | NULL |
| 4 | COMPLETED | Eve | 2024-03-10 | 2024-04-20 |
| 5 | Cloud Migration | FRANK | 2024-03-15 | NULL |
+-----+-----+-----+-----+
5 rows in set (0.00 sec)
```

```
mysql> DELETE FROM PROJECTS WHERE ASSIGNEDTO IS NULL;
Query OK, 0 rows affected (0.01 sec)
```

```
mysql> DELETE FROM PROJECTS WHERE START IS NULL;
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> DELETE FROM PROJECTS WHERE END IS NOT NULL;
Query OK, 2 rows affected (0.00 sec)
```

```
mysql> SELECT* FROM PROJECTS
-> ;
+-----+-----+-----+-----+-----+
| id | name | assignedto | start | end |
+-----+-----+-----+-----+
| 2 | Mobile App | Bob | 2024-02-01 | NULL |
| 3 | Data Migration | FRANK | 2026-07-23 | NULL |
| 5 | Cloud Migration | FRANK | 2024-03-15 | NULL |
+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

```
mysql> -- STEP 1: Create Table
mysql> CREATE TABLE Orders (
-> id INT,
-> customer_name VARCHAR(100),
-> order_date DATE,
-> total_amount DECIMAL(10,2)
-> );
Query OK, 0 rows affected (0.02 sec)
```

```
mysql>
mysql> -- STEP 2: Insert Records
mysql> INSERT INTO Orders VALUES
-> (1, 'Alice', '2024-03-01', 1500.00),
-> (2, 'Bob', '2024-03-05', 450.00),
-> (3, 'Charlie', '2024-03-10', 2500.00),
-> (4, 'Diana', '2024-03-15', 800.00),
-> (5, 'Eve', '2024-03-20', 1200.00),
-> (6, 'Frank', '2024-03-25', 3000.00),
-> (7, 'Grace', '2024-04-02', 1800.00),
-> (8, 'Henry', '2024-04-08', 950.00),
-> (9, 'Ivy', '2024-04-12', 2750.00),
-> (10, 'Jack', '2024-04-18', 650.00),
-> (11, 'Kevin', '2024-04-22', 4200.00),
-> (12, 'Lily', '2024-05-01', 1350.00),
-> (13, 'Michael', '2024-05-08', 5000.00),
```

```
-> (14, 'Nancy', '2024-05-15', 2250.00),
-> (15, 'Oliver', '2024-05-25', 900.00);
Query OK, 15 rows affected (0.01 sec)
Records: 15 Duplicates: 0 Warnings: 0
```

```
mysql> SELECT*FROM ORDERS;
+-----+-----+-----+-----+
| id | customer_name | order_date | total_amount |
+-----+-----+-----+-----+
| 1 | Alice | 2024-03-01 | 1500.00 |
| 2 | Bob | 2024-03-05 | 450.00 |
| 3 | Charlie | 2024-03-10 | 2500.00 |
| 4 | Diana | 2024-03-15 | 800.00 |
| 5 | Eve | 2024-03-20 | 1200.00 |
| 6 | Frank | 2024-03-25 | 3000.00 |
| 7 | Grace | 2024-04-02 | 1800.00 |
| 8 | Henry | 2024-04-08 | 950.00 |
| 9 | Ivy | 2024-04-12 | 2750.00 |
| 10 | Jack | 2024-04-18 | 650.00 |
| 11 | Kevin | 2024-04-22 | 4200.00 |
| 12 | Lily | 2024-05-01 | 1350.00 |
| 13 | Michael | 2024-05-08 | 5000.00 |
| 14 | Nancy | 2024-05-15 | 2250.00 |
| 15 | Oliver | 2024-05-25 | 900.00 |
+-----+-----+-----+-----+
15 rows in set (0.00 sec)
```

```
mysql> SELECT* FROM ORDERS WHERE ORDER_DATE BETWEEN '2024-03-03' AND '2024-03-10';
+-----+-----+-----+-----+
| id | customer_name | order_date | total_amount |
+-----+-----+-----+-----+
| 2 | Bob | 2024-03-05 | 450.00 |
| 3 | Charlie | 2024-03-10 | 2500.00 |
+-----+-----+-----+-----+
2 rows in set (0.01 sec)
```

```
mysql> SELECT* FROM ORDERS WHERE ORDER_DATE BETWEEN '2024-04-01' AND '2024-04-30';
+-----+-----+-----+-----+
| id | customer_name | order_date | total_amount |
+-----+-----+-----+-----+
| 7 | Grace | 2024-04-02 | 1800.00 |
| 8 | Henry | 2024-04-08 | 950.00 |
| 9 | Ivy | 2024-04-12 | 2750.00 |
| 10 | Jack | 2024-04-18 | 650.00 |
| 11 | Kevin | 2024-04-22 | 4200.00 |
+-----+-----+-----+-----+
5 rows in set (0.00 sec)
```

```
mysql> SELECT* FROM ORDERS ORDER_DATE BETWEEN '2024-04-01' AND '2024-04-30' WHERE TOTAL_AMOUNT=TOTAL_AMOUNT+1000;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'BETWEEN '2024-04-01' AND '2024-04-30' WHERE TOTAL_AMOUNT=TOTAL_AMOUNT+1000' at line 1
mysql> SELECT* FROM ORDERS ORDER_DATE BETWEEN '2024-03-04' AND '2024-03-10' WHERE TOTAL_AMOUNT=TOTAL_AMOUNT+1000;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'BETWEEN '2024-03-04' AND '2024-03-10' WHERE TOTAL_AMOUNT=TOTAL_AMOUNT+1000' at line 1
mysql> UPDATE ORDERS SET ORDER_DATE BETWEEN '2024-03-04' AND '2024-03-10' WHERE TOTAL_AMOUNT=TOTAL_AMOUNT+1000;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'BETWEEN '2024-03-04' AND '2024-03-10' WHERE TOTAL_AMOUNT=TOTAL_AMOUNT+1000' at line 1
mysql> UPDATE ORDERS SET ORDER_DATE BETWEEN '2024-03-04' AND '2024-03-10' WHERE TOTAL_AMOUNT=TOTAL_AMOUNT+1000 ORDER_DATE BETWEEN '2024-03-04' AND '2024-03-10';
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'BETWEEN '2024-03-04' AND '2024-03-10' WHERE TOTAL_AMOUNT=TOTAL_AMOUNT+1000 ORDER' at li
mysql> UPDATE ORDERS SET WHERE TOTAL_AMOUNT=TOTAL_AMOUNT+1000 WHERE ORDER_DATE BETWEEN '2024-03-04' AND '2024-03-10';
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'WHERE TOTAL_AMOUNT=TOTAL_AMOUNT+1000 WHERE ORDER_DATE BETWEEN '2024-03-04' AND '' at li
mysql> UPDATE ORDERS SET WHERE TOTAL_AMOUNT=TOTAL_AMOUNT+1000 WHERE ORDER_DATE BETWEEN '2024-03-04' AND '2024-03-10';
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'WHERE TOTAL_AMOUNT=TOTAL_AMOUNT+1000 WHERE ORDER_DATE BETWEEN '2024-03-04' AND '' at li
mysql> ^C
mysql> UPDATE ORDERS SET TOTAL_AMOUNT=TOTAL_AMOUNT+1000 WHERE ORDER_DATE BETWEEN '2024-03-04' AND '2024-03-10';
Query OK, 2 rows affected (0.01 sec)
Rows matched: 2 Changed: 2 Warnings: 0
```

```
mysql> UPDATE ORDERS SET TOTAL_AMOUNT= TOTAL_AMOUNT+(TOTAL_AMOUNT*0.10) WHERE TOTAL_AMOUNT BETWEEN 2500 AND 5000;
Query OK, 5 rows affected (0.01 sec)
Rows matched: 5 Changed: 5 Warnings: 0
```

```
mysql> DELETE FROM ORDERS WHERE TOTAL_AMOUNT BETWEEN 2500 AND 5000;
Query OK, 4 rows affected (0.01 sec)
```

```
mysql> SELECT*FROM ORDERS
-> ;
+-----+-----+-----+-----+
| id | customer_name | order_date | total_amount |
+-----+-----+-----+-----+
| 1 | Alice | 2024-03-01 | 1500.00 |
| 2 | Bob | 2024-03-05 | 1450.00 |
| 4 | Diana | 2024-03-15 | 800.00 |
| 5 | Eve | 2024-03-20 | 1200.00 |
| 7 | Grace | 2024-04-02 | 1800.00 |
| 8 | Henry | 2024-04-08 | 950.00 |
```

```
| 10 | Jack      | 2024-04-18 | 650.00 |
| 12 | Lily      | 2024-05-01 | 1350.00 |
| 13 | Michael   | 2024-05-08 | 5500.00 |
| 14 | Nancy     | 2024-05-15 | 2250.00 |
| 15 | Oliver    | 2024-05-25 | 900.00 |
+-----+
11 rows in set (0.00 sec)
```

```
mysql> DELETE FROM ORDERS WHERE ORDER_DATE BETWEEN '2024-03-10' AND '2024-03-20';
Query OK, 2 rows affected (0.01 sec)
```

```
mysql> SELECT * FROM ORDERS;
+-----+
| id | customer_name | order_date | total_amount |
+-----+
| 1  | Alice         | 2024-03-01 | 1500.00      |
| 2  | Bob           | 2024-03-05 | 1450.00      |
| 7  | Grace         | 2024-04-02 | 1800.00      |
| 8  | Henry         | 2024-04-08 | 950.00       |
| 10 | Jack          | 2024-04-18 | 650.00       |
| 12 | Lily          | 2024-05-01 | 1350.00      |
| 13 | Michael       | 2024-05-08 | 5500.00      |
| 14 | Nancy         | 2024-05-15 | 2250.00      |
| 15 | Oliver        | 2024-05-25 | 900.00       |
+-----+
9 rows in set (0.00 sec)
```

```
mysql> -- STEP 1: Create Table
mysql> CREATE TABLE Emp (
  -> id INT,
  -> name VARCHAR(100),
  -> dept VARCHAR(20),
  -> salary INT
  -> );
Query OK, 0 rows affected (0.04 sec)
```

```
mysql>
mysql> -- STEP 2: Insert Records
mysql> INSERT INTO Emp VALUES
  -> (1, 'Alice', 'HR', 35000),
  -> (2, 'Bob', 'IT', 50000),
  -> (3, 'Charlie', 'Finance', 45000),
  -> (4, 'Diana', 'Marketing', 40000),
  -> (5, 'Eve', 'IT', 55000),
  -> (6, 'Frank', 'Finance', 48000),
  -> (7, 'Grace', 'Sales', 37000),
  -> (8, 'Henry', 'Admin', 42000),
  -> (9, 'Ivy', 'HR', 39000),
  -> (10, 'Jack', 'Sales', 46000),
  -> (11, 'Kevin', 'IT', 60000),
  -> (12, 'Lily', 'Marketing', 41000),
  -> (13, 'Michael', 'Admin', 38000),
  -> (14, 'Nancy', 'Finance', 52000),
  -> (15, 'Oliver', 'HR', 43000);
Query OK, 15 rows affected (0.01 sec)
Records: 15 Duplicates: 0 Warnings: 0
```

```
mysql>
mysql> SELECT*FROM EMP;
+-----+
| id | name | dept | salary |
+-----+
| 1  | Alice | HR   | 35000  |
| 2  | Bob   | IT   | 50000  |
| 3  | Charlie | Finance | 45000  |
| 4  | Diana | Marketing | 40000  |
| 5  | Eve   | IT   | 55000  |
| 6  | Frank | Finance | 48000  |
| 7  | Grace | Sales | 37000  |
| 8  | Henry | Admin | 42000  |
| 9  | Ivy   | HR   | 39000  |
| 10 | Jack  | Sales | 46000  |
| 11 | Kevin | IT   | 60000  |
| 12 | Lily  | Marketing | 41000  |
| 13 | Michael | Admin | 38000  |
| 14 | Nancy | Finance | 52000  |
| 15 | Oliver | HR   | 43000  |
+-----+
15 rows in set (0.00 sec)
```

```
mysql> SELECT* FROM EMP WHERE DEPT IN('IT','FINANCE');
+-----+
| id | name | dept | salary |
```

```
+-----+-----+-----+
| 2 | Bob | IT | 50000 |
| 3 | Charlie | Finance | 45000 |
| 5 | Eve | IT | 55000 |
| 6 | Frank | Finance | 48000 |
| 11 | Kevin | IT | 60000 |
| 14 | Nancy | Finance | 52000 |
+-----+-----+-----+
```

6 rows in set (0.01 sec)

```
mysql> UPDATE EMP SET SALARY=SALARY+2000 WHERE DEPT IN ('HR','MARKETING');
Query OK, 5 rows affected (0.01 sec)
Rows matched: 5 Changed: 5 Warnings: 0
```

```
mysql> DELETE FROM EMP WHERE DEPT IN('FINANCE','MARKETING');
Query OK, 5 rows affected (0.01 sec)
```

```
mysql> DELETE FROM EMP WHERE DEPT IN('SALES','IT');
Query OK, 5 rows affected (0.01 sec)
```

```
mysql> DELETE FROM EMP WHERE DEPT IN('ADMIN','HR');
Query OK, 5 rows affected (0.01 sec)
```

```
mysql> SELECT*from emp;
Empty set (0.00 sec)
```

mysql> STEP 2: Insert Records

```
-> INSERT INTO Emp VALUES
-> (1, 'Rahul', 'IT', 50000),
-> (2, 'Ramesh', 'HR', 35000),
-> (3, 'Karan', 'Finance', 45000),
-> (4, 'Mahesh', 'Marketing', 40000),
-> (5, 'Suresh', 'IT', 55000),
-> (6, 'Arjun', 'Sales', 38000),
-> (7, 'Manish', 'Admin', 42000),
-> (8, 'Ganesh', 'Finance', 47000),
-> (9, 'Naresh', 'HR', 39000),
-> (10, 'Rohan', 'IT', 52000),
-> (11, 'Mohan', 'Sales', 41000),
-> (12, 'Kishan', 'Marketing', 43000),
-> (13, 'Dinesh', 'Admin', 46000),
-> (14, 'Rajesh', 'Finance', 51000),
-> (15, 'Kiran', 'IT', 48000);
```

ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'STEP 2: Insert Records

```
INSERT INTO Emp VALUES
(1, 'Rahul', 'IT', 50000),
(2, '' at line 1
```

```
mysql>
mysql> select*from emp;
Empty set (0.00 sec)
```

mysql> ^C

```
mysql> INSERT INTO Emp VALUES
-> (1, 'Rahul', 'IT', 50000),
-> (2, 'Ramesh', 'HR', 35000),
-> (3, 'Karan', 'Finance', 45000),
-> (4, 'Mahesh', 'Marketing', 40000),
-> (5, 'Suresh', 'IT', 55000),
-> (6, 'Arjun', 'Sales', 38000),
-> (7, 'Manish', 'Admin', 42000),
-> (8, 'Ganesh', 'Finance', 47000),
-> (9, 'Naresh', 'HR', 39000),
-> (10, 'Rohan', 'IT', 52000),
-> (11, 'Mohan', 'Sales', 41000),
-> (12, 'Kishan', 'Marketing', 43000),
-> (13, 'Dinesh', 'Admin', 46000),
-> (14, 'Rajesh', 'Finance', 51000),
-> (15, 'Kiran', 'IT', 48000);
```

```
Query OK, 15 rows affected (0.01 sec)
Records: 15 Duplicates: 0 Warnings: 0
```

```
mysql> select*from emp;
```

```
+-----+-----+-----+
| id | name | dept | salary |
+-----+-----+-----+
| 1 | Rahul | IT | 50000 |
| 2 | Ramesh | HR | 35000 |
| 3 | Karan | Finance | 45000 |
| 4 | Mahesh | Marketing | 40000 |
| 5 | Suresh | IT | 55000 |
| 6 | Arjun | Sales | 38000 |
| 7 | Manish | Admin | 42000 |
| 8 | Ganesh | Finance | 47000 |
```

9	Naresh	HR	39000
10	Rohan	IT	52000
11	Mohan	Sales	41000
12	Kishan	Marketing	43000
13	Dinesh	Admin	46000
14	Rajesh	Finance	51000
15	Kiran	IT	48000

15 rows in set (0.00 sec)

```
mysql> select*from emp where like 'r%';
```

ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'like 'r%'' at line 1

```
mysql> SELECT* FROM EMP WHERE NAME LIKE 'R%';
```

id	name	dept	salary
1	Rahul	IT	50000
2	Ramesh	HR	35000
10	Rohan	IT	52000
14	Rajesh	Finance	51000

4 rows in set (0.00 sec)

```
mysql> SELECT* FROM EMP WHERE NAME LIKE '%SH';
```

id	name	dept	salary
2	Ramesh	HR	35000
4	Mahesh	Marketing	40000
5	Suresh	IT	55000
7	Manish	Admin	42000
8	Ganesh	Finance	47000
9	Naresh	HR	39000
13	Dinesh	Admin	46000
14	Rajesh	Finance	51000

8 rows in set (0.00 sec)

```
mysql> SELECT* FROM EMP WHERE NAME LIKE '%AR%';
```

id	name	dept	salary
3	Karan	Finance	45000
6	Arjun	Sales	38000
9	Naresh	HR	39000

3 rows in set (0.00 sec)

```
mysql> SELECT* FROM EMP WHERE NAME LIKE '____';
```

id	name	dept	salary
1	Rahul	IT	50000
3	Karan	Finance	45000
6	Arjun	Sales	38000
10	Rohan	IT	52000
11	Mohan	Sales	41000
15	Kiran	IT	48000

6 rows in set (0.00 sec)

```
mysql> SELECT* FROM EMP WHERE NAME LIKE 'K%N';
```

id	name	dept	salary
3	Karan	Finance	45000
12	Kishan	Marketing	43000
15	Kiran	IT	48000

3 rows in set (0.00 sec)

```
mysql> SELECT* FROM EMP WHERE NAME LIKE 'M%E%';
```

id	name	dept	salary
4	Mahesh	Marketing	40000

1 row in set (0.00 sec)

```
mysql> SELECT* FROM EMP WHERE NAME LIKE '-----H';
```

Empty set (0.00 sec)

```
mysql> SELECT* FROM EMP WHERE NAME LIKE '____H';
```

```
+-----+
| id | name | dept | salary |
+-----+
| 2 | Ramesh | HR | 35000 |
| 4 | Mahesh | Marketing | 40000 |
| 5 | Suresh | IT | 55000 |
| 7 | Manish | Admin | 42000 |
| 8 | Ganesh | Finance | 47000 |
| 9 | Naresh | HR | 39000 |
| 13 | Dinesh | Admin | 46000 |
| 14 | Rajesh | Finance | 51000 |
+-----+
8 rows in set (0.00 sec)
```

```
mysql> SELECT* FROM EMP WHERE NAME LIKE 'A%E%I%O%U%';
Empty set (0.00 sec)
```

```
mysql> SELECT* FROM EMP WHERE NAME LIKE 'A%'OR'E%'OR'I%'OR 'O%'OR'U%';
+-----+
| id | name | dept | salary |
+-----+
| 6 | Arjun | Sales | 38000 |
+-----+
1 row in set, 4 warnings (0.01 sec)
```

```
mysql> SELECT* FROM EMP WHERE NAME NOT LIKE 'A%'OR'E%'OR'I%'OR 'O%'OR'U%';
+-----+
| id | name | dept | salary |
+-----+
| 1 | Rahul | IT | 50000 |
| 2 | Ramesh | HR | 35000 |
| 3 | Karan | Finance | 45000 |
| 4 | Mahesh | Marketing | 40000 |
| 5 | Suresh | IT | 55000 |
| 7 | Manish | Admin | 42000 |
| 8 | Ganesh | Finance | 47000 |
| 9 | Naresh | HR | 39000 |
| 10 | Rohan | IT | 52000 |
| 11 | Mohan | Sales | 41000 |
| 12 | Kishan | Marketing | 43000 |
| 13 | Dinesh | Admin | 46000 |
| 14 | Rajesh | Finance | 51000 |
| 15 | Kiran | IT | 48000 |
+-----+
14 rows in set, 4 warnings (0.01 sec)
```

```
mysql> CREATE TABLE employees (
-> emp_id INT PRIMARY KEY,
-> emp_name VARCHAR(50),
-> department VARCHAR(30),
-> salary DECIMAL(10,2),
-> experience INT,
-> age INT
-> );
ERROR 1050 (42S01): Table 'employees' already exists
```

```
mysql>
mysql> INSERT INTO employees VALUES
-> (101, 'Ravi', 'IT', 65000, 5, 28),
-> (102, 'Priya', 'HR', 45000, 2, 24),
-> (103, 'Amit', 'Finance', 75000, 8, 35),
-> (104, 'Sneha', 'IT', 55000, 3, 27),
-> (105, 'Rahul', 'Sales', 40000, 1, 22),
-> (106, 'Kiran', 'Finance', 90000, 12, 40),
-> (107, 'Anjali', 'HR', 52000, 4, 29),
-> (108, 'Suresh', 'Sales', 68000, 7, 33);
ERROR 1136 (21S01): Column count doesn't match value count at row 1
mysql> DROP TABLES EMPLOYEES;
Query OK, 0 rows affected (0.04 sec)
```

```
mysql> CREATE TABLE employees (
-> emp_id INT PRIMARY KEY,
-> emp_name VARCHAR(50),
-> department VARCHAR(30),
-> salary DECIMAL(10,2),
-> experience INT,
-> age INT
-> );
Query OK, 0 rows affected (0.03 sec)
```

```
mysql>
mysql> INSERT INTO employees VALUES
-> (101, 'Ravi', 'IT', 65000, 5, 28),
-> (102, 'Priya', 'HR', 45000, 2, 24),
```

```

-> (103,'Amit','Finance',75000,8,35),
-> (104,'Sneha','IT',55000,3,27),
-> (105,'Rahul','Sales',40000,1,22),
-> (106,'Kiran','Finance',90000,12,40),
-> (107,'Anjali','HR',52000,4,29),
-> (108,'Suresh','Sales',68000,7,33);

```

Query OK, 8 rows affected (0.01 sec)

Records: 8 Duplicates: 0 Warnings: 0

```
mysql> SELECT* FROM EMPLOYEES;
```

emp_id	emp_name	department	salary	experience	age
101	Ravi	IT	65000.00	5	28
102	Priya	HR	45000.00	2	24
103	Amit	Finance	75000.00	8	35
104	Sneha	IT	55000.00	3	27
105	Rahul	Sales	40000.00	1	22
106	Kiran	Finance	90000.00	12	40
107	Anjali	HR	52000.00	4	29
108	Suresh	Sales	68000.00	7	33

8 rows in set (0.01 sec)

```

mysql> SELECT*,
-> CASE
-> ^C
mysql> SELECT EMP_NAME, SALARY,
-> CASE
-> WHEN SALARY>=80000 THEN 'HIGH_SALARY'
-> WHEN SALARY>=60000 THEN 'LOW_SALARY'
-> ELSE 'LOW_SALARY'
-> END AS SALARY_CATEGORY
-> FROM EMPLOYEES;

```

EMP_NAME	SALARY	SALARY_CATEGORY
Ravi	65000.00	LOW_SALARY
Priya	45000.00	LOW_SALARY
Amit	75000.00	LOW_SALARY
Sneha	55000.00	LOW_SALARY
Rahul	40000.00	LOW_SALARY
Kiran	90000.00	HIGH_SALARY
Anjali	52000.00	LOW_SALARY
Suresh	68000.00	LOW_SALARY

8 rows in set (0.01 sec)

```

mysql> SELECT EMP_NAME, EXPERIENCE,
-> CASE
-> WHEN EXPERIENCE>=10 THEN 'EXPERT'
-> WHEN EXPERIENCE>=5 THEN 'SENIOR'
-> WHEN EXPERIENCE>=2 THEN 'JUNIOR FRESHER'
-> ELSE 'UNEMPLOYED'
-> END AS EMP_CATEGORY
-> FROM EMPLOYEES;

```

EMP_NAME	EXPERIENCE	EMP_CATEGORY
Ravi	5	SENIOR
Priya	2	JUNIOR FRESHER
Amit	8	SENIOR
Sneha	3	JUNIOR FRESHER
Rahul	1	UNEMPLOYED
Kiran	12	EXPERT
Anjali	4	JUNIOR FRESHER
Suresh	7	SENIOR

8 rows in set (0.00 sec)

```

mysql> SELECT EMP_NAME, AGE,
-> WHEN AGE>=18 THEN 'ELIGIBLE TO VOTE'
-> ELSE 'INELIGIBLE TO VOTE'
-> END AS VOTERS
-> FROM EMPLOYEES;

```

ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'WHEN AGE>=18 THEN 'ELIGIBLE TO VOTE'

ELSE 'INELIGIBLE TO VOTE'

END AS VOTERS

FRO' at line 2

MySQL Day 10 - Interview Questions & Answers

Q1. Why did `SELECT NAME,ASSIGNEDTO WHERE ASSIGNEDTO='NULL';` fail with 'Unknown column NAME in field list'?

A: The statement is missing the FROM clause entirely (no table specified), so MySQL cannot resolve NAME or ASSIGNEDTO as columns of any table. A valid SELECT needs FROM table_name before the WHERE clause.

Q2. Why did `SELECT * FROM PROJECTS WHERE ASSIGNEDTO='NULL';` return an empty set even though some rows have NULL assignedto?

A: Here 'NULL' is written as a quoted string literal, so MySQL compares ASSIGNEDTO to the text "NULL", not to the actual SQL NULL marker. Since no row literally contains the string 'NULL', the query returns no matches -- the comparison operator = can never match true SQL NULL values.

Q3. Why did `WHERE ASSIGNEDTO IS 'NULL';` throw a syntax error, and what is the correct way to test for NULL?

A: IS must be followed by the keyword NULL (or NOT NULL), not a quoted string; IS 'NULL' is invalid syntax. The correct form is WHERE ASSIGNEDTO IS NULL, which uses the special NULL-testing operator rather than an equality comparison.

Q4. Why can't you use = or != to check for NULL values in SQL?

A: In SQL's three-valued logic, NULL represents an unknown value, so any comparison involving NULL with =, !=, <, or > evaluates to UNKNOWN rather than TRUE or FALSE, and such rows are excluded from results. Only the IS NULL / IS NOT NULL operators are designed to correctly test for the presence or absence of NULL.

Q5. What does `UPDATE PROJECTS SET START = NOW() WHERE START IS NULL;` do, and why did it report '1 warning'?

A: NOW() returns the current date and time, so this fills the empty START column with today's date/time for rows where it was NULL. The warning appears because START is a DATE column while NOW() returns a DATETIME; MySQL truncates the time portion to fit the DATE type, which triggers a data-truncation warning even though the operation succeeds.

Q6. Why did `UPDATE PROJECTS SET END='COMPLETED' WHERE END IS NOT NULL;` fail with 'Incorrect date value'?

A: END is defined as a DATE column, and 'COMPLETED' is not a valid date format. MySQL enforces the column's data type, so attempting to store a non-date string in a DATE column raises Error 1292 instead of silently accepting it.

Q7. What is the purpose of the BETWEEN operator, as in `WHERE ORDER_DATE BETWEEN '2024-03-03' AND '2024-03-10'?`

A: BETWEEN selects rows where the column's value falls within an inclusive range, matching any date from the start bound through the end bound (both included). It is equivalent to writing `ORDER_DATE >= '2024-03-03' AND ORDER_DATE <= '2024-03-10'`, just more concise.

Q8. Why did `UPDATE ORDERS SET WHERE TOTAL_AMOUNT=TOTAL_AMOUNT+1000 WHERE ...` fail repeatedly before the correct version worked?

A: The SET clause mistakenly included the keyword WHERE before the column assignment, and separately the query tried to use BETWEEN directly after the table name without a WHERE clause, which UPDATE's syntax doesn't allow. The fix was `UPDATE table SET column=expression WHERE condition`, keeping SET and WHERE in their proper positions with only one WHERE.

Q9. What is the difference between the IN operator and multiple OR conditions, as seen in `WHERE DEPT IN('IT','FINANCE')`?

A: IN(...) checks whether a column's value matches any value in a given list, functioning as shorthand for `DEPT='IT' OR DEPT='FINANCE'`. It is more concise and readable than chaining several OR comparisons, especially with longer lists.

Q10. What does the % wildcard do in a LIKE pattern, as in WHERE NAME LIKE 'R%' versus '%SH'?

A: % matches zero or more of any character. 'R%' matches names starting with R (e.g., Rahul, Ramesh), while '%SH' matches names ending in 'SH' (e.g., Ramesh, Suresh), since % can absorb any leading or trailing characters.

Q11. What does the _ wildcard do in a LIKE pattern, and why did WHERE NAME LIKE '_____' return only 5-letter names?

A: Unlike %, the underscore _ matches exactly one single character. Five consecutive underscores therefore requires exactly five characters total, so only names with precisely 5 letters (like Rahul, Karan, Arjun) matched.

Q12. Why did SELECT * FROM EMP WHERE NAME LIKE 'A%'OR'E%'OR'I%'OR 'O%'OR'U%'; return only 1 row and '4 warnings' instead of matching names starting with any vowel?

A: Each OR'd term after the first is just a bare string literal, not a NAME LIKE comparison, so MySQL implicitly coerces each string ('E%', 'I%', etc.) into a boolean truth value rather than applying LIKE to it. Only the first condition (NAME LIKE 'A%') is meaningful; the rest are truthy/falsy literals, which is why MySQL raised 4 warnings and the result only reflects the 'A%' match.

Q13. Why did CREATE TABLE employees (...) fail the first time with 'Table employees already exists', succeed after DROP TABLE, but the INSERT before the drop fail with 'Column count doesn't match value count'?

A: An 'employees' table (lowercase) already existed from an earlier session with a different structure, so CREATE TABLE failed, and INSERT INTO employees VALUES(...) failed because the number of values per row (7 fields) didn't match the old table's actual column count. Dropping the old table and recreating it with the intended 6-column definition resolved both issues.

Q14. What does a CASE expression do, as used to build the SALARY_CATEGORY column?

A: CASE evaluates a list of WHEN condition THEN result branches in order and returns the result of the first matching condition, or the ELSE value if none match; it functions like an inline if/elif/else that can be aliased as a computed column in the SELECT list.

Q15. In the SALARY_CATEGORY query, both WHEN SALARY>=60000 and the ELSE branch returned 'LOW_SALARY'. Was that likely intentional?

A: It appears to be a labeling mistake: with only 'HIGH_SALARY' (>=80000) and two branches both producing 'LOW_SALARY', there is no distinct MID/LOW tier, so salaries from 60000-79999 and salaries below 60000 are indistinguishable in the output, even though the CASE structure suggests three intended tiers.

Q16. Why did SELECT EMP_NAME,AGE, WHEN AGE>=18 THEN ... END AS VOTERS FROM EMPLOYEES; fail with a syntax error?

A: The CASE keyword itself was omitted before the WHEN clauses; a CASE expression must begin with the literal keyword CASE, followed by one or more WHEN...THEN branches and an END. Without CASE, MySQL doesn't recognize WHEN as valid syntax in that position.

Q17. What is the difference between DROP TABLE and DROP TABLES as used in the transcript?

A: DROP TABLES (plural) is simply MySQL's accepted alternate keyword form for dropping one or more tables in a single statement; both DROP TABLE and DROP TABLES work for dropping a single table, and DROP TABLES additionally allows a comma-separated list of multiple table names in one command.